

Introduction to ROS - Autonomous Driving

1st Bhaskar Sah

*Department of Mechanical Engineering
Technical University of Munich
Munich, Germany
bhaskar.sah@tum.de*

2nd Ramkrishna Chaudhari

*Department of Mechanical Engineering
Technical University of Munich
Munich, Germany
ramkrishna.chaudhari@tum.de*

3rd Muayad Chalabi

*Department of Mechanical Engineering
Technical University of Munich
Munich, Germany
muayad.chalabi@tum.de*

4th Youssef Ibrahim

*Department of Mechanical Engineering
Technical University of Munich
Munich, Germany
go52xac@mytum.de*

1 Introduction

This paper documents the final Autonomous Driving project of group 15 in the course **Introduction to ROS** at Autonomous Aerial System (AAS) of TU Munich. The main challenge of the project is to develop a robust autonomous vehicle system capable of safely navigating through the environments developed in Unity using perception, planning, and control modules integrated into a ROS-based software stack. The group members and the distribution of workload can be seen below:

Ramkrishna Chaudhari: Perception (perception_pkg), Traffic light detection (traffic_rule_pkg) & State machine (state_machine_pkg)

Muayad Chalabi: Global path planning (global_planner)

Bhaskar Sah: Local path planning (local_planner_pkg), Trajectory planning & Control (trajectory_planner_and_control_pkg)

Youssef Ibrahim: Global path planning (global_planner), Controller testing and Stanley Controller attempt

2 Project Architecture

The overall system architecture for the autonomous driving project is illustrated in Figure 1. It demonstrates the modular integration of perception, planning, decision-making, and control subsystems within the ROS framework. Each component communicates through standardized ROS topics, ensuring scalability and maintainability.

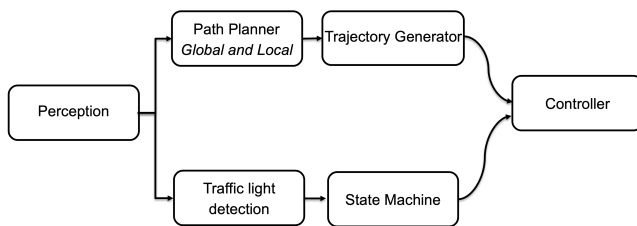


Figure 1: System architecture of the autonomous driving project showing the interaction between perception, planning, control, and decision-making modules.

This architecture emphasizes a clear separation of con-

cerns: the perception pipeline processes raw sensor data into structured environment representations; the planning modules (global and local) use this information to generate safe trajectories; the state machine governs decision logic such as traffic light handling; and the control module translates planned trajectories into actionable commands for vehicle actuation. This modular design facilitates debugging, parallel development. Furthermore, the interconnection of ROS nodes and their communication channels is represented in the ROS computation graph, generated using `rqt_graph`. This graph illustrates the data flow across perception, planning, control, and decision-making modules, explicitly depicting published and subscribed topics among nodes. For clarity and readability, the detailed ROS graph is presented in Appendix A.

3 Perception Pipeline

The `perception_system` forms the sensory processing backbone of the autonomous vehicle, transforming raw sensor inputs into actionable environmental representations. Built upon the Robot Operating System (ROS) framework, the architecture employs a nodelet-based design for efficient data processing. At its core resides a `nodelet_manager` instance that orchestrates all perception tasks, enabling lightweight intra-process communication while minimizing computational overhead.

The system processes four primary sensor streams: a depth camera, stereo RGB cameras, and a semantic camera. Each sensor feed undergoes initial geometric rectification to correct lens distortion artifacts. This preprocessing stage utilizes specialized `image_proc/rectify` nodelets that apply camera-specific calibration parameters to produce distortion-corrected images. The depth camera stream follows a dual processing pathway - while the rectified images are available for direct use, they also feed into a point cloud generation pipeline.

For three-dimensional environment modeling, the system employs a `depth_image_proc/point_cloud_xyz` nodelet that transforms rectified depth images into dense point cloud representations. These point clouds serve two distinct purposes: a direct feed for real-time obstacle detection, and a processed stream dedicated to volumetric mapping. The latter feeds into an `octomap_server` node that

constructs and maintains a persistent 3D occupancy map of the environment. Configured with 0.7 meter voxel resolution and a 20 meter sensing range, this occupancy map provides the spatial awareness necessary for navigation tasks while remaining computationally tractable.

The perception pipeline's outputs support multiple autonomous functions. Rectified image streams enable visual perception tasks, while the point cloud data facilitates immediate obstacle detection. The volumetric occupancy map serves higher-level path planning and decision-making modules. This multi-output architecture successfully balances the competing demands of real-time responsiveness and comprehensive environment modeling, forming a critical component of the vehicle's autonomous capabilities.

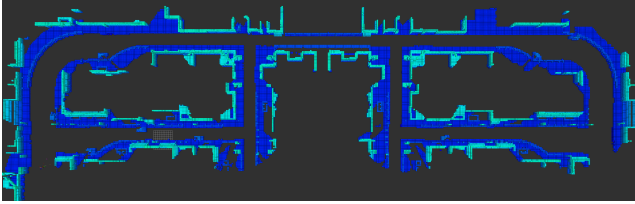


Figure 2: 3D occupancy map generated using the octomap_server node, illustrating the volumetric mapping of the environment in Rviz.

4 Traffic Light Detection System

The traffic light detection module represents a critical perception component for urban autonomous driving scenarios. Implemented as the `rgb_camera_decoder` ROS node, this system integrates multi-modal sensor data to achieve robust traffic light state classification with distance estimation. The architecture combines RGB appearance data, depth information, and semantic segmentation in a sensor fusion pipeline.

4.1 Sensor Inputs and Preprocessing

The system processes three synchronized input streams:

$$I_{input} = \{I_{rgb}, I_{depth}, I_{semantic}\} \quad (1)$$

where I_{rgb} denotes the RGB image (BGR8 encoding), I_{depth} contains depth measurements (16UC1), and $I_{semantic}$ provides pixel-wise classifications. All inputs are received as ROS `sensor_msgs/Image` messages through dedicated topics:

- RGB: `/Unity_ROS_message_Rx/OurCar/Sensors/RGBCameraRight/image_Rect`
- Depth: `/Unity_ROS_message_Rx/OurCar/Sensors/DepthCamera/image_Rect`
- Semantic: `/Unity_ROS_message_Rx/OurCar/Sensors/SemanticCamera/image_Rect`

Each input undergoes conversion to OpenCV matrices via `cv_bridge`, with explicit checks for message encoding compatibility. The system enforces temporal synchro-

nization through a readiness flag mechanism, only initiating processing when all three sensor inputs are refreshed.

4.2 Detection Algorithm

The core detection pipeline operates in four stages:

4.2.1 Semantic Masking

First, the semantic image $I_{semantic}$ is resized to match I_{rgb} dimensions using nearest-neighbor interpolation, preserving class boundaries:

$$I_{semantic}^{resized} = \text{resize}(I_{semantic}, \text{size}(I_{rgb}), \text{INTER_NEAREST}) \quad (2)$$

A binary mask M is then generated through thresholding at intensity value 210:

$$M(x, y) = \begin{cases} 1 & \text{if } I_{semantic}^{resized}(x, y) \geq 210 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

4.2.2 Region of Interest Extraction

The system defines an ROI R centered in the upper image portion where traffic lights are statistically most probable:

$$R = \left(\frac{W}{2} - \frac{W}{2r_w}, 0, \frac{W}{r_w}, \frac{H}{r_h} \right) \quad (4)$$

where W, H represent image dimensions, and $r_w = 3, r_h = 1$ are configurable size ratios.

4.2.3 Color Classification

Each pixel $p \in R$ is classified through multi-threshold analysis:

$$\text{Red: } p_r > p_g + t_r \wedge p_r > p_b \quad (5)$$

$$\text{Yellow: } p_r > t_y \wedge p_g > t_y \wedge p_b < \frac{t_y}{2} \quad (6)$$

$$\text{Green: } p_g > p_r + t_g \wedge p_g > p_b \quad (7)$$

where $t_r = 150, t_y = 140, t_g = 150$ are configurable thresholds.

4.2.4 Distance Estimation

For each classified pixel, corresponding depth values from I_{depth} are accumulated and averaged:

$$d_{avg}^c = \frac{\sum_{p \in P_c} I_{depth}(p)}{|P_c|} \quad (8)$$

where $c \in \{\text{red}, \text{yellow}, \text{green}\}$ and P_c denotes pixels of class c .

4.3 Output Interface

The traffic light detection node publishes three synchronized output streams to communicate critical detection results to downstream systems. The primary output `Is_TrafficLight_RED` (`std_msgs/Bool`) provides a boolean stop signal, where a `true` value indicates an immediate stopping requirement and `false` permits continued movement. This low-latency binary signal serves as direct input for the vehicle's safety-critical braking system.

For state-aware decision making, the node generates `TrafficLight_State` (`std_msgs/String`), encoding the classified light state as one of three discrete values: "RED", "YELLOW", or "GREEN". This categorical output enables the planning system to implement appropriate control policies, including anticipatory slowing for yellow lights and intersection traversal timing for green phases.

The distance estimation output `dist_2_TrafficLight` (`std_msgs/Int32`) provides metric distance measurements to the detected traffic signal, enabling precise velocity planning. The distance calculation follows:

$$d = \frac{1}{n} \sum_{i=1}^n z_i \quad [\text{meters}] \quad (9)$$

where z_i represents valid depth measurements for classified traffic light pixels and n is the sample count. This continuous-valued output supports smooth deceleration profiles when approaching intersections.

These outputs collectively form a comprehensive detection interface that satisfies both safety requirements through the boolean stop signal and planning needs via the state-distance pair. The design ensures reliable operation across diverse urban scenarios while maintaining compatibility with standard ROS message types for system integration.

The configurable parameters (thresholds, ROI ratios) enable adaptation to various operational environments while maintaining real-time performance at 30Hz on our test platform.

5 Global Path Planning

The `global_planner_node.py` constitutes the core component of the vehicle's high-level path planning module. Its primary function is to compute a complete global path prior to vehicle operation, based on a predefined set of nine waypoints and a known occupancy grid map of the environment. The occupancy grid, published on the `/global_map` topic, is supplied by a dedicated map server node that loads a previously generated global map. This approach assumes full prior knowledge of the environment and simplifies the computational complexity of online planning by avoiding the need for incremental updates during vehicle operation.

The global map is generated offline using a `map_fuzer` node, which aggregates and merges local occupancy grids produced by the Octomap server into a unified global occupancy representation. This fused map is subsequently stored in a `my_global_map.pgm` file. The map server node then loads this stored map and makes it accessible to the planning system by publishing it as an occupancy grid over the

`/global_map` topic. This offline generation and subsequent use of a static global map allows the planner to operate under the assumption of a fully observable environment, which reduces computational overhead compared to online mapping and planning frameworks such as Simultaneous Localization and Mapping (SLAM). While SLAM-based approaches are more flexible in unstructured or unknown environments, the static-map paradigm adopted here is suitable for the constrained simulation setting.

Upon initialization, the `global_planner_node` subscribes to both the vehicle's current pose and the global occupancy grid. Using this information, it computes an optimal global path that traverses the predefined waypoints while avoiding obstacles encoded in the occupancy grid. The resulting path is represented as a continuous sequence of poses and is published as a `nav_msgs::Path` message on the `/global_planner` topic. In addition, a look-ahead point on the path is continuously identified and published on the `/global_planner.point` topic to support downstream trajectory tracking and control modules.

The `map_fuzer` node operates as a mapping component that continuously integrates local projected occupancy maps, generated by the Octomap server, with the ego vehicle's pose estimates. Through spatial alignment and incremental fusion, it constructs a fixed-size global map that accurately represents the environment. This fused map is periodically saved and subsequently reloaded by the map server node, thereby decoupling the mapping process from the planning and control layers of the system. The map server node itself functions as a static data provider, loading stored maps from the repository and publishing them to ensure a consistent environmental representation during planning.

Collectively, this architecture establishes a global planning pipeline in which environmental knowledge is first consolidated into a global occupancy grid and then exploited by the `global_planner_node` to generate an obstacle-free reference path. This design, while reliant on the availability of a pre-mapped environment, significantly reduces online computational demands and facilitates stable and reproducible path planning in the simulation framework. It also establishes a clear functional separation between mapping, global planning, and local control, aligning with best practices in modular robotics software design.



Figure 3: Generated global path visualized on the fused occupancy grid map using A* search algorithm.

5.1 Path Finding Algorithm Details

The path planning algorithm used in this project is a generic Python implementation of the A* (A-star) search algorithm. To ensure that the resulting path is suitable for autonomous navigation, the following preprocessing steps are applied:

1. **Occupancy Grid Dilation:** The occupancy grid received from the map server is dilated using a morphological kernel of size greater than 3×3 . This ensures that the planned path maintains a safe buffer zone—at least one grid cell—away from actual obstacles. The physical size of each grid cell (in meters) is determined by the Octomap resolution parameter.
2. **Cost Map Generation:** A cost map is computed based on the distance to the nearest occupied cell. This encourages the algorithm to favor paths that pass through the center of drivable space, reducing the likelihood of undesirable behaviors such as navigating too close to walls or obstacles (commonly referred to as "wall hugging").

6 Local Path Planning: Tactical Obstacle Avoidance

The `local_planner_node`, implemented as part of the `local_planner_pkg` package, plays a crucial role in enabling reactive, short-term path planning for autonomous vehicles operating in dynamic environments. It is designed to continuously compute safe and drivable local waypoints by analyzing a 3D representation of the surrounding environment provided by OctoMap. Unlike the global planner, which follows a precomputed path, the local planner responds to real-time changes, such as unexpected obstacles or moving entities.

Upon receiving the current vehicle pose and a global waypoint as reference, the node evaluates multiple candidate directions within a predefined angular span around the vehicle's heading. This span forms a fan-shaped arc that is incrementally sampled in both clockwise and counter-clockwise directions from the forward direction. For each sampled direction, the planner projects a candidate point at a configurable distance ahead. These candidate points are checked for occupancy using the OctoMap, which encodes obstacle information in a probabilistic 3D grid.

Mathematically, the projection of each candidate point $\vec{p}_i$ is computed as:

$$\vec{p}_i = \vec{p}_{curr} + d \cdot \begin{bmatrix} \cos(\theta + \phi_i) \\ \sin(\theta + \phi_i) \end{bmatrix}, \quad \phi_i \in [-\alpha, \alpha] \quad (10)$$

where $\vec{p}_{curr}$ is the current position of the vehicle, d is the lookahead distance, θ is the current orientation, and ϕ_i is the angular offset for each ray.

The planner checks each projected point in sequence and selects the first one that lies in a free, collision-free region. This approach ensures minimal delay in decision-making and allows the vehicle to swiftly adapt its path in response to dynamic elements in the environment. If no valid direction is found, the node withholds publishing a new waypoint, signaling the controller to initiate deceleration or stopping, thus providing an inherent safety fallback.

Tunable parameters exposed via ROS allow developers to adapt the planner's behavior without recompiling the code. These include `distanceCheck`, which sets the lookahead distance; `widthCheck`, which ensures a safety buffer around candidate paths; and `searchResolution`, which defines the angular granularity of the fan search.

In essence, the `local_planner_node` bridges the gap between global navigation and reactive control. By maintaining a continuous loop of environment scanning and safe waypoint selection, it ensures that the vehicle can navigate safely even in the presence of dynamic and previously unknown obstacles. Its modular design and ROS interface make it a critical and easily extendable component in any autonomous driving stack.

The node also visualizes the evaluated search region in RViz by publishing bounding boxes, aiding debugging and tuning. The fan-shaped search expands symmetrically around the vehicle's heading, checking both right and left deviations incrementally to ensure minimal steering adjustments while maintaining safety.

7 Trajectory Planning and Control: From Plan to Action

The `trajectoryPlanner_and_Control_node`, implemented within the `trajectoryPlanner_and_Control_pkg` package, is responsible for translating the output of the local planner into actionable vehicle control commands, such as steering, acceleration, and braking. This node forms the final link in the perception-planning-control pipeline and directly interfaces with the vehicle's actuation systems. Its goal is to ensure that the car can follow the dynamically updated local waypoint while maintaining stability, comfort, and safety.

Conceptually, the node implements a low-level feedback control strategy, integrating both lateral and longitudinal motion control. It does so by constantly comparing the desired vehicle state, determined from the lookahead point published by the local planner, with the current vehicle state obtained from odometry and localization.

7.1 Inputs and Outputs

- **Inputs:** Lookahead point, current pose and speed, and high-level behavior from `/state_machine/current.state`
- **Output:** Vehicle actuation commands on `/car_command`

7.2 Steering Control: P Controller

To control the steering angle, a proportional (P) controller is used as describe in [7]. The controller computes the angular error θ_{error} between the vehicle's current heading and the heading to the lookahead point. This is calculated using the arctangent of the relative position of the target point. The control law is expressed as:

$$\theta_{error} = \theta_{target} - \theta_{current}, \quad (11)$$

$$\delta = K_p \cdot \theta_{error}, \quad (12)$$

where δ is the steering angle command and K_p is a tunable gain parameter. This simple yet effective approach ensures that the vehicle continuously aligns its heading with the target direction. While more advanced lateral controllers such as pure pursuit or Stanley are available, the P-controller remains effective for many structured environments and allows quick tuning.

7.3 Speed Control: PID Controller

For longitudinal speed control, a PID controller is implemented as described in [7]. This controller calculates the acceleration or deceleration required to match the vehicle's current speed with the target speed associated with the lookahead point. The error term $e(t)$ is defined as:

$$e(t) = v_{target} - v(t), \quad (13)$$

leading to the control law:

$$a(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}, \quad (14)$$

where $a(t)$ is the acceleration command. Positive values of $a(t)$ translate to throttle commands, while negative values result in braking. Proper tuning of the PID parameters is essential to avoid oscillations, overshoot, or sluggish behavior.

7.4 State Machine Integration

In addition to continuous control, the node is designed to be behavior-aware. It listens to the finite state machine (FSM) via the `/state_machine/current_state` topic to adapt its behavior based on high-level driving contexts. For instance, in the `DRIVING` state, it operates the full PID and steering control loop. However, in states such as `SLOW`, `STOP` or `PARKED`, the node overrides normal control with predefined braking logic through the `handleStop()` routine. This separation of concerns between motion control and decision-making ensures cleaner architecture and easier debugging.

Overall, the `trajectoryPlannerAndControl_node` encapsulates the logic required to safely and smoothly guide the vehicle along a trajectory. By combining real-time feedback control with behavioral state awareness, it forms a robust foundation for low-level vehicle autonomy. Its simplicity, modularity, and clean interfaces make it a practical yet scalable solution for autonomous driving applications. This flexibility and modularity of the controller struck the interest to try and further build on the controller by attempting a more challenging control algorithm namely "Stanley control" after testing the original PID controller as presented in Appendix B.

8 State Machine

The `state_machine_node` serves as the central decision-making component for autonomous vehicle navigation, managing transitions between distinct operational states based on environmental inputs. The system begins in an `IDLE` state during initialization, maintaining this mode for

precisely 5 seconds to ensure proper system readiness before transitioning to the primary `DRIVING` state.

During normal operation in the `DRIVING` state, the node follows path from `global_planner` while continuously monitoring for two critical conditions: proximity to special navigation points and traffic light status. When approaching within 30 meters of designated special waypoints, the system transitions to the `SLOW` state, reducing velocity in preparation for potential stopping requirements. The `SLOW` state implements more nuanced behavior, maintaining reduced speed until either reaching the target waypoint (within 10 meter tolerance) and reverting to `DRIVING`, or encountering a confirmed red light at closer range (8-15 meters) which triggers transition to `STOP`. The stop condition persists until the traffic light changes to green or the secondary confirmation is invalidated, at which point normal `DRIVING` resumes.

Final transition to the `PARKED` state occurs automatically upon successful completion of all route waypoints, representing mission accomplishment. Key operational parameters include a 10 meter waypoint acceptance threshold, 30 meter slowdown initiation distance, and 10Hz state evaluation frequency, ensuring responsive yet stable behavior. The implementation emphasizes safety through redundant condition verification and carefully tuned spatial thresholds that accommodate both traffic regulations and vehicle dynamics. The state machine is illustrated in Figure 4.

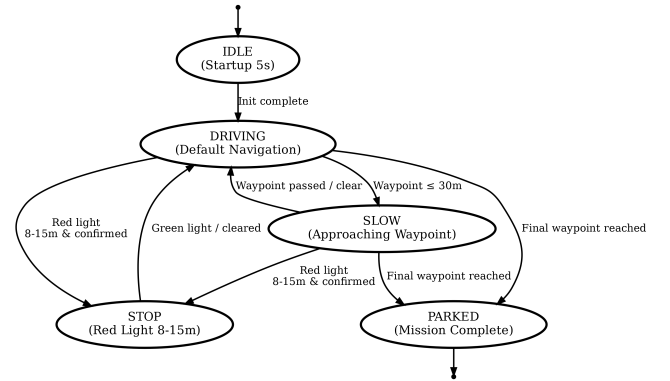


Figure 4: State machine diagram illustrating transitions between `IDLE`, `DRIVING`, `SLOW`, `STOP`, and `PARKED` states based on waypoint proximity and traffic light conditions.

9 Conclusion and Overview

This project successfully demonstrated the development of a ROS-based autonomous driving system integrating perception, planning, and control within a simulated urban environment. Through a modular architecture, we implemented a robust perception pipeline leveraging depth, RGB, and semantic sensors to enable 3D occupancy mapping and traffic light detection. The planning framework combined a static global path planner with a reactive local planner for obstacle avoidance, complemented by a trajectory control module employing PID-based speed regulation and proportional steering control.

Decision-making was orchestrated through a state ma-

chine that managed behaviors such as traffic light compliance and speed modulation near critical waypoints. Furthermore, an alternative global planner utilizing OMPL-RRT* was developed. Although its computational cost precluded real-time integration, it demonstrated strong potential for future use in optimal path planning.

The system successfully achieved its primary objective of autonomously navigating the Parkour track while responding effectively to dynamic events, with an average lap completion time of approximately 239 seconds. The modular design, adherence to ROS development best practices, and effective system integration establish a strong foundation for future extensions, including real-world deployment, dynamic mapping with SLAM, and the incorporation of advanced planning and control strategies such as MPC.

References

1. A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard. "OctoMap: An efficient probabilistic 3D mapping framework based on octrees." *Autonomous Robots*, 2013. doi:10.1007/s10514-012-9321-0
2. ROS Wiki: *depth_image_proc*. Available at: http://wiki.ros.org/depth_image_proc
3. ROS Wiki: *OctoMap*. Available at: <http://wiki.ros.org/octomap>
4. OpenCV Developers. *Image Thresholding in OpenCV*. Available at: https://docs.opencv.org/3.4/d7/d4d/tutorial_py_thresholding.html
5. Adrian Rosebrock. *Image Masking with OpenCV*. Available at: <https://pyimagesearch.com/2021/01/19/image-masking-with-opencv/>
6. OpenCV Team. *Image Thresholding Using OpenCV*. Available at: <https://opencv.org/blog/image-thresholding-using-opencv/>
7. Wikipedia contributors. *Proportional–integral–derivative controller*. In: *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller (Accessed: July 28, 2025).

Appendix

A ROS Graph Visualization

Figure 5 illustrates the ROS computation graph generated using `rqt_graph`. It shows all nodes and their topic connections, including perception nodes handling sensor data (RGB, depth, and semantic cameras), the planning nodes (`global_planner` and `local_planner`), control nodes for vehicle actuation, and the state machine coordinating decision-making.

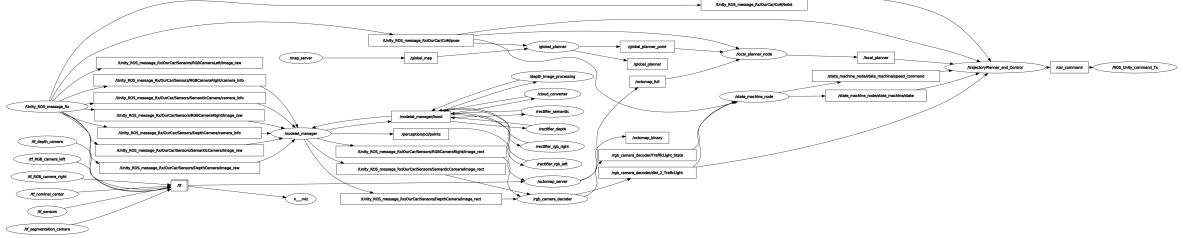


Figure 5: ROS computation graph generated using `rqt_graph`

This figure highlights the modularity of the ROS ecosystem, showing how data flows from perception nodes to planning modules, is supervised by the state machine, and finally reaches the control system to generate actuation commands. It confirms the proper topic-node communication and validates the integration of all project components in a cohesive ROS framework.

B Stanley Controller Attempt

To try and further improve the accuracy and stability of lateral motion, especially in curved paths and sharp turns, a Stanley controller was explored by **Youssef Ibrahim** as a more robust alternative to the proportional controller. What motivated this is that the Stanley controller is widely used in autonomous driving due to its effectiveness in minimizing both Heading error (difference in orientation), and Cross-track error (lateral distance to the path). Its control law is typically defined as:

$$\delta = \theta_{\text{error}} + \tan^{-1} \left(\frac{k \cdot e_{\text{ct}}}{v + \epsilon} \right) \quad (15)$$

where:

- δ : steering angle command,
- θ_{error} : heading error,
- e_{ct} : cross-track error,
- v : current speed,
- k : tuning gain,
- ϵ : small constant to avoid division by zero.

The implementation involved calculating the nearest waypoint on the path relative to the car's current position; computing heading and cross-track errors based on the vehicle's current pose relative to the path; and deriving steering commands using the Stanley formula.

Despite successfully implementing the mathematical structure of the controller, several integration challenges were encountered including incorrect or unstable steering behavior where the vehicle exhibited aggressive or oscillating turns due to overestimated cross-track error. There were also speed-dependency issues since the Stanley controller scales its correction with speed, at low speeds the control became erratic due to division instability.

Although the Stanley controller was not finalized in the final pipeline, the attempt demonstrated the flexibility and modularity of the system, showcasing how more advanced control techniques can be integrated with minimal structural changes. Future work may focus on refining this controller, possibly combining it with filtering mechanisms to improve robustness.

B.1 Controller Testing and Validation

To ensure that the implemented control logic behaves reliably across various scenarios, a series of tests were conducted in simulation as well as in isolation without wrapping it in ROS. These tests evaluated how the `trajectoryPlanner_and_Control` node responds to dynamically changing waypoints, variations in target speed, and transitions between behavioral states. Testing involved:

- Driving along straight and curved segments, where the lateral and longitudinal controllers were monitored for stability, responsiveness, and smoothness where graphs were plotted as well to monitor the behavior.
- Reacting to stop commands issued by the `/state_machine/current_state` topic to validate behavior switching logic.
- Parameter tuning in real-time through ROS parameters for gains like K_p , K_i , and K_d , helping to eliminate oscillations and improve convergence speed.

Results from testing demonstrated that:

- The PID controller provides effective longitudinal control under moderate speeds.
- The proportional steering controller can track the lookahead waypoint in structured environments.
- The state machine transitions such as `STOP`, `SLOW` or `PARKED` were correctly honored, and braking behavior was appropriately implemented during these states.

These evaluations provided both validation of functionality and insights into limitations, motivating the exploration of more advanced steering controllers.

C Alternative: Global Path Planner Node

An extra `global_planner_node.cpp` was developed by **Ramkrishna Chaudhari** as an alternative global path planning solution for the autonomous driving Project. This node employed the Open Motion Planning Library (OMPL) and implemented the Rapidly-exploring Random Tree Star RRT-STAR algorithm for optimal path generation in continuous state spaces. Although the node was fully functional and generated collision-free, dynamically feasible paths during evaluation, it was ultimately excluded from the final project integration due to its computational demands in real-time execution and the need for minor tuning of planner parameters such as sampling density, goal bias, and planning time. Nevertheless, the planner effectively demonstrated a powerful and principled approach to global path planning suitable for integration in more advanced versions of the project.

RRT-STAR is an asymptotically optimal sampling-based motion planning algorithm. It incrementally builds a search tree rooted at the vehicle's start position by sampling random states within the defined configuration space and extending the tree toward these states. Unlike the basic RRT algorithm, RRT-STAR incorporates a rewiring step that iteratively improves path quality by reconnecting new samples to lower-cost neighboring nodes. This ensures that as the number of samples increases, the solution converges to an optimal path with respect to a specified cost function. In this implementation, the cost was defined by OMPL's `PathLengthOptimizationObjective` and augmented with a custom safety term that penalized proximity to obstacles detected in the occupancy map, thereby encouraging safer and smoother trajectories.

The planning process began by defining a two-dimensional state space representing the drivable road area, derived from map constraints and vehicle kinematics. A state validity checker was integrated using OctoMap to ensure that sampled states lay within collision-free regions of the environment. The planner then iteratively sampled candidate states, extended the tree using vehicle-aware steering constraints, and rewired the resulting nodes to minimize cost. Once a node was extended sufficiently close to the goal, OMPL's solution extraction mechanisms reconstructed the optimal path. This path was post-processed using OMPL's built-in shortcutting and spline interpolation methods to remove unnecessary waypoints and ensure smoothness suitable for downstream trajectory tracking.

This node subscribed to the ego vehicle's current pose, the goal pose defined within the simulation map, and the environment's occupancy representation derived from OctoMap. The final optimized trajectory was published as a `nav_msgs::Path` message and visualized in RViz using `visualization_msgs::MarkerArray` for validation and debugging. By employing OMPL's modular interface, the node provided clear separation between state space definition, validity checking, optimization objectives, and planning, reflecting a structured and reusable design.

The main reason this node was not deployed in the final integrated project was its runtime cost during high-frequency replanning scenarios. RRT-STAR requires a sufficient number of iterations to approach optimality, and although feasible paths were found quickly, achieving smooth near-optimal solutions required computation times beyond the strict real-time constraints of this project. However, in all tests where sufficient planning time was permitted, the node successfully generated valid and safe global paths. Its integration of OMPL and RRT-STAR represents a scientifically rigorous and modular foundation for future research-oriented or higher-performance extensions of the planning pipeline.

In the context of this project, this planner demonstrates how global planning can be achieved using state-of-the-art sampling-based motion planning rather than simple waypoint-following. Its combination of `RRT-STAR` with OctoMap-based collision checking highlights the capability to operate in complex and unstructured road networks while maintaining generalizability to other planning problems beyond urban driving. Although omitted from the final implementation, it provides a robust reference for incorporating optimal motion planning into ROS-based autonomous driving architectures and remains a significant contribution to the project's development.